🔥 CATEGORY 11: Tricky Troubleshooting, Edge Cases & Interview

> **Difficulty:** Intermediate → Advanced | **Topics:** 20 | **Terraform Interview Mastery Series**

Table of Contents

1. Phantom Apply Debugging
2. State Drift Reconciliation
3. Concurrent Apply Conflicts
4. Cross-State Resource Migration
5. Stuck State Lock Recovery
6. Silent Drift
7. Perpetual Diff Debugging
8. Plan Passes But Apply Fails
9. `(known after apply)` Cascading
10. Safe Resource Renaming
11. `count` to `for_each` Live Migration
12. Root Module to Child Module Refactoring
13. Third-Party Module Breaking Change Handling
14. Accidental Secret Commit Recovery
15. Sensitive Value Leak Tracing
16. Retroactive State Encryption
17. Slow Plan Debugging
18. Large State File Management
19. Parallelism Tuning
20. Provider Version Conflict Resolution

Topic 88: Phantom Apply Debugging

🔵 What It Is (Simple Terms)

A **phantom apply** is when `terraform apply` exits with success — exit code 0, "Apply complete!" message — but the resource either doesn't exist in the cloud or exists in a different state than expected. The apply lied to you.

🔵 Why It Happens — Root Causes

Cause 1: Async Resource Creation

```
terraform apply completes
  → Provider calls CreateInstance API
  → AWS returns 200 OK (accepted, not completed)
  → Terraform marks resource as created ✅
  → Terraform exits

Background (minutes later):
  → AWS actually provisions the instance
  → OR the provisioning fails silently

Result: State says "i-abc123 exists", AWS has no such instance
```

Cause 2: Provider Bug — Incorrect Error Handling

```
# Some providers swallow errors from the API
# They log a warning but return success to Terraform Core
# Terraform writes to state as if creation succeeded

# Common with:
# - Third-party/community providers
# - AWS resources with eventual consistency (IAM, Route53)
# - Resources that require multi-step creation
```

Cause 3: Wrong Account/Region

```
# Provider pointed at wrong region/account
provider "aws" {
  region = "us-east-1"   # ← applied here
}

# But you're looking in us-west-2
# Apply succeeded — resource IS there — just not where you're looking
```

Cause 4: State Written Before API Completes

```
Apply sequence:
  1. Terraform calls Create API → gets resource ID back immediately
  2. Terraform writes ID to state ← state says "created"
  3. Resource is still initializing in AWS (pending state)
  4. Initialization fails (wrong AMI, insufficient capacity, etc.)
  5. AWS deletes the resource

Result: State has the ID, AWS has deleted it
```

🔵 How to Debug

```
# Step 1: Check what Terraform thinks exists
terraform state list
terraform state show aws_instance.web

# Step 2: Check what actually exists in AWS
aws ec2 describe-instances --instance-ids i-abc123
# If not found: "InvalidInstanceID.NotFound"

# Step 3: Run refresh to detect the gap
terraform plan -refresh=true
# Will show: "aws_instance.web must be replaced" or "will be created"
# Because refresh detects the resource is gone

# Step 4: Check provider logs
export TF_LOG=DEBUG
export TF_LOG_PATH=terraform-debug.log
terraform apply 2>&1 | tee apply-output.log
# Look for: HTTP response codes, API error messages

# Step 5: Check AWS CloudTrail
# CloudTrail shows every API call — look for CreateInstance
# and any subsequent errors (DeleteInstance, etc.)
```

🔵 Recovery Steps

```
# If resource genuinely doesn't exist but state says it does:

# Option 1: Remove from state and recreate
terraform state rm aws_instance.web
terraform apply   # will recreate

# Option 2: Run plan — Terraform will detect it's missing
terraform plan
# Shows: "aws_instance.web will be created" (because refresh found it
gone)
terraform apply   # recreates it

# Option 3: If resource exists but in wrong place/account
# Fix the provider config → re-run plan → state will reconcile
```

🔵 Prevention

```
# Use timeouts to catch async failures
resource "aws_db_instance" "main" {
  identifier = "prod-db"
  # ...

  timeouts {
    create = "40m"   # Wait up to 40 min for creation
    update = "80m"
    delete = "40m"
  }
}

# Always verify after apply in CI/CD
# Add a post-apply validation step:
# aws ec2 describe-instances --instance-ids $(terraform output
instance_id)
```

🔵 Short Interview Answer

> "A phantom apply happens when Terraform reports success but the resource doesn't exist. Common causes: async API calls where AWS accepts the request but provisioning fails after Terraform exits, provider bugs that swallow errors, or Terraform writing state after getting an ID back from the API before the resource finishes initializing. Debug with `terraform plan -refresh=true` which will detect the resource is gone and show it as needing creation. In production, always add post-apply validation steps in CI/CD that verify critical resources actually exist after apply."

🔵 Tricky Interview Questions

Q: `terraform apply` shows "Apply complete! 1 added." but `terraform plan` immediately after shows "1 to add." How?

> "The resource was created in state but doesn't exist in the cloud. `terraform plan` runs a refresh — it calls the AWS API to check the resource's actual state. When it can't find the resource (it was never really created, or was deleted asynchronously), it marks it as needing creation again. This is a phantom apply scenario. Fix: `terraform plan -refresh=true` to confirm, then `terraform apply` again, or `terraform state rm` and reapply."

Topic 89: State Drift Reconciliation

🔵 What It Is (Simple Terms)

State drift is when the real world diverges from what Terraform's state file believes. Someone manually changed an EC2 instance type in the console, deleted a security group rule, or added a tag — without going

through Terraform. Now state and reality disagree.

🔵 Types of Drift

```
Type 1: ATTRIBUTE drift
  State says:  instance_type = "t3.medium"
  Reality:     instance_type = "t3.large"  (manually changed)
  Plan shows:  ~ instance_type = "t3.large" -> "t3.medium"  (Terraform
wants to revert)

Type 2: EXISTENCE drift — resource deleted outside Terraform
  State says:  aws_security_group.web exists
  Reality:     security group was manually deleted
  Plan shows:  + aws_security_group.web will be created  (Terraform
recreates it)

Type 3: EXTRA resource drift — resource created outside Terraform
  State says:  nothing about this S3 bucket
  Reality:     S3 bucket "manual-bucket" exists
  Plan shows:  nothing  (Terraform doesn't know about it — invisible
drift)
```

🔵 How Terraform Detects Drift

```
# terraform plan always refreshes state by default
# It calls provider APIs to check current attribute values

terraform plan
# Internally:
# 1. Read state → get resource IDs
# 2. Call AWS API: DescribeInstances, DescribeSecurityGroups etc.
# 3. Compare API response to state
# 4. If different → show as change in plan

# Fast refresh (skips API calls — use only in known-stable environments)
terraform plan -refresh=false

# Explicit refresh without planning
terraform apply -refresh-only        # Terraform 1.1+ — update state to
match reality
terraform refresh                    # Deprecated but still works
```

🔵 The `refresh-only` Apply — The Right Tool

```
# Scenario: lots of manual changes were made, you want to
# ACCEPT them into state without reverting

# Step 1: See what drifted
terraform plan -refresh-only
# Shows all differences between state and reality
# Does NOT show what would be changed to match config

# Step 2: Accept reality into state
terraform apply -refresh-only
# Updates state file to match current real-world state
# Config is NOT changed — just state is updated

# Step 3: Now plan shows what config wants vs current reality
terraform plan
# Now you see the full picture: what Terraform would change
# to bring reality back to config
```

🔵 Reconciliation Decision Tree

```
Drift detected. What do you do?

Was the manual change intentional?
  YES → Should you codify it?
    YES → Update .tf config to match reality → terraform plan should be
clean
    NO  → Run terraform apply to revert the manual change back to config

Was the manual change an emergency fix?
  YES → Was it to fix a production incident?
    YES → Update the .tf config to match → commit → apply to confirm
    NO  → Revert with terraform apply

Is the resource managed by another team and you're just reading it?
  → Use data source instead of resource block — you shouldn't own it
```

🔵 Short Interview Answer

> "State drift is when reality diverges from Terraform's state — someone manually changed something outside Terraform. There are three types: attribute drift (same resource, different config), existence drift (resource deleted manually), and invisible drift (resource created manually that Terraform doesn't know about). `terraform plan` detects drift via refresh — it calls provider APIs to compare current state against what's in the state file. In Terraform 1.1+, `terraform apply -refresh-only` lets you accept manual changes into state without reverting them — useful when the manual change was intentional and you want to codify it going forward."

🔵 Gotchas & Edge Cases

- ⚠ `-refresh=false` **hides drift** — fast but dangerous. Use only when you're certain no manual changes occurred.
- ⚠ **Type 3 drift (unknown resources) is invisible** — Terraform has no way to detect resources it doesn't know about. Use AWS Config or cloud inventory tools for full drift detection.
- `refresh-only` **doesn't fix your config** — it only updates state. After `refresh-only`, your config still has the old values. You must manually update `.tf` files to codify the change.

Topic 90: Concurrent Apply Conflicts

🔵 What It Is (Simple Terms)

What happens when two engineers (or two CI/CD pipelines) run `terraform apply` at the exact same time against the same state file? This is a real production risk in teams without proper workflow controls.

🔵 State Locking — The Defense Mechanism

```
terraform apply starts:
  1. Acquires state lock (writes lock record to DynamoDB / TFC / GCS)
  2. Reads current state
  3. Creates plan
  4. Applies changes
  5. Writes new state
  6. Releases lock

Second apply tries to start simultaneously:
  1. Tries to acquire lock
  2. Lock is held by first apply
  3. BLOCKED with error:

Error: Error acquiring the state lock
  Error message: ConditionalCheckFailedException
  Lock Info:
    ID:        abc123
    Path:      mycompany-tf-state/prod/terraform.tfstate
    Operation: OperationTypeApply
    Who:       engineer@laptop
    Version:   1.5.0
    Created:   2024-01-15 10:30:00
```

🔵 S3+DynamoDB Locking Setup

```
terraform {
  backend "s3" {
    bucket         = "mycompany-terraform-state"
    key            = "prod/main/terraform.tfstate"
    region         = "us-east-1"
    encrypt        = true

    # DynamoDB table for state locking
    dynamodb_table = "terraform-state-locks"
    # Table must have: primary key = "LockID" (String)
  }
}
```

```
# Create the DynamoDB lock table
aws dynamodb create-table \
  --table-name terraform-state-locks \
  --attribute-definitions AttributeName=LockID,AttributeType=S \
  --key-schema AttributeName=LockID,KeyType=HASH \
  --billing-mode PAY_PER_REQUEST
```

🔵 What Happens WITHOUT Locking

```
Engineer A applies at 10:00:00:
  Reads state → 5 resources
  Creates EC2 instance (i-aaa)
  Writes state → 6 resources (includes i-aaa)

Engineer B applies at 10:00:01 (before A finishes):
  Reads state → 5 resources (before A wrote new state)
  Creates EC2 instance (i-bbb)
  Writes state → 6 resources (overwrites A's state!)

Result:
  State has 6 resources (missing i-aaa — A's instance lost from state)
  Reality has 7 resources (i-aaa and i-bbb both exist)
  State corruption — i-aaa is now an orphan
```

🔵 Prevention Strategies

```
1. Remote backend with locking (S3+DynamoDB, TFC, GCS) — MANDATORY
2. PR-based workflow — only one apply can run per merge
3. Terraform Cloud — serializes all runs per workspace
4. Atlantis — queues applies, one at a time per repo
```

```
5. Branch protection — prevent direct pushes to main
6. CI/CD serialization — use job concurrency controls
```

🔵 Short Interview Answer

> "Concurrent applies are prevented by state locking. When apply starts, Terraform writes a lock record to DynamoDB (for S3 backend), GCS, or Terraform Cloud. Any concurrent apply attempt reads the lock, sees it's held, and fails immediately with a detailed error showing who holds the lock and when it was acquired. Without locking — like with a local state file or improperly configured remote backend — two concurrent applies can result in state corruption where one apply's changes are silently overwritten. In team environments, always use a remote backend with locking enabled."

Topic 91: Cross-State Resource Migration

🔵 What It Is (Simple Terms)

Moving a resource from one Terraform state file to another — without destroying and recreating it. This comes up when reorganizing infrastructure, splitting monolithic configs, or moving resources between teams.

🔵 The Problem

```
State A manages: VPC + Subnets + EC2 + RDS
You want:        State A = VPC + Subnets
                 State B = EC2 + RDS

Naive approach:
  Remove from State A config → terraform apply → DESTROYS EC2 and RDS
  Add to State B config → terraform apply → RECREATES EC2 and RDS
  Result: DATA LOSS on RDS, downtime on EC2
```

🔵 The Right Approach: `terraform state mv`

```
# Method 1: terraform state mv (classic approach)

# Step 1: Pull both state files locally
terraform state pull > state-a.tfstate         # from workspace A

# Step 2: Move resource from state A to state B
terraform state mv \
```

```
  -state=state-a.tfstate \
  -state-out=state-b.tfstate \
  aws_instance.web \
  aws_instance.web

# Step 3: Move RDS too
terraform state mv \
  -state=state-a.tfstate \
  -state-out=state-b.tfstate \
  aws_db_instance.main \
  aws_db_instance.main

# Step 4: Push updated states back
cd workspace-a && terraform state push state-a.tfstate
cd workspace-b && terraform state push state-b.tfstate

# Step 5: Update .tf configs
# Remove resources from workspace A config
# Add resources to workspace B config

# Step 6: Verify
cd workspace-a && terraform plan  # should show no changes to moved
resources
cd workspace-b && terraform plan  # should show no changes (resources
exist in state)
```

🔵 The Modern Approach: `moved` Block + State Push

```
# In the destination module/config — Terraform 1.1+
# This is cleaner for same-state refactoring but limited for cross-state

moved {
  from = aws_instance.web
  to   = module.app.aws_instance.web
}
```

🔵 Full Safe Migration Checklist

```
Pre-migration:
  ✅ Take state backups of both state files
  ✅ Document all resources being moved
  ✅ Identify all outputs/references that will break
  ✅ Plan the config changes in both modules
  ✅ Test in a non-production environment first

Migration:
  ✅ Disable CI/CD applies (prevent concurrent changes)
```

```
    ✅ Move state entries
    ✅ Update source config (remove resources)
    ✅ Update destination config (add resources)
    ✅ Run plan on source → verify 0 changes for moved resources
    ✅ Run plan on destination → verify 0 changes for moved resources

  Post-migration:
    ✅ Re-enable CI/CD
    ✅ Update cross-stack references (remote_state, SSM params)
    ✅ Update documentation
    ✅ Monitor for 24 hours
```

🔵 Short Interview Answer

> "Cross-state migration moves resources between state files without destroying them. The process: pull both state files, use `terraform state mv -state=source.tfstate -state-out=dest.tfstate` to move each resource's state entry, push the updated states back, then update the `.tf` configs — remove from source, add to destination. Run plan on both — if done correctly, both show zero changes because the resources are in state and match config. The critical step people miss is backing up both state files first. One mistake here can result in state corruption or orphaned resources."

Topic 92: Stuck State Lock Recovery

🔵 What It Is (Simple Terms)

A state lock that won't release. The apply that held it crashed, the CI job was killed, the laptop died mid-apply — and now the lock record is stuck in DynamoDB/GCS/TFC. Every subsequent apply fails with "state is locked."

🔵 How Locks Get Stuck

```
  Normal lock lifecycle:
    acquire lock → apply → release lock

  Stuck lock scenarios:
    1. Process killed (SIGKILL, kill -9, job timeout)
    2. Network failure between Terraform and backend
    3. Terraform panic/crash during apply
    4. CI/CD job timed out and was killed
    5. Machine powering off during apply
```

🔵 Diagnosing a Stuck Lock

```
# Error you'll see:
Error: Error acquiring the state lock
  Error message: ConditionalCheckFailedException
  Lock Info:
    ID:        f3b45c2d-1234-5678-abcd-ef0123456789
    Path:      mycompany-tf-state/prod/terraform.tfstate
    Operation: OperationTypeApply
    Who:       ci-runner@github-actions
    Version:   1.5.0
    Created:   2024-01-15 02:30:00 UTC    ← lock from 6 hours ago —
definitely stuck

# Verify: Is the apply actually still running?
# Check CI/CD dashboard — is there an active job?
# Check who "ci-runner@github-actions" was — was it a failed pipeline?
```

🔵 Recovery: `terraform force-unlock`

```
# ONLY after confirming no apply is actually running
terraform force-unlock f3b45c2d-1234-5678-abcd-ef0123456789

# With -force flag to skip confirmation prompt (for automation)
terraform force-unlock -force f3b45c2d-1234-5678-abcd-ef0123456789

# Success message:
# Terraform state has been successfully unlocked!
```

🔵 Direct DynamoDB Recovery (Emergency)

```
# If terraform force-unlock itself fails (backend connectivity issue):

# Find the lock record
aws dynamodb get-item \
  --table-name terraform-state-locks \
  --key '{"LockID": {"S": "mycompany-tf-state/prod/terraform.tfstate"}}'

# Manually delete the lock record
aws dynamodb delete-item \
  --table-name terraform-state-locks \
  --key '{"LockID": {"S": "mycompany-tf-state/prod/terraform.tfstate"}}'

# ⚠ ONLY do this if you are 100% certain no apply is running
```

🔵 The Golden Rule of Force-Unlock

```
Before force-unlocking, answer ALL of these:

1. Is the apply that held the lock definitely NOT running?
   → Check CI/CD, check all terminals, check all team members

2. Is the state file in a consistent state?
   → Check if the last apply completed any resource changes
   → If apply was mid-flight, state may be partially written

3. Is it safe to allow new applies?
   → Will a new apply pick up from where the stuck one left off?
   → Run terraform plan first after unlocking to assess state
```

🔵 Short Interview Answer

> "State locks get stuck when the process holding the lock crashes or is killed — CI job timeout, machine power loss, network failure. Recovery is `terraform force-unlock <lock-id>` where the lock ID comes from the error message. Before running this, you must be 100% certain the apply that held the lock is no longer running — if it is still running and you force-unlock, a second apply can start and you'll have concurrent writes to state. If `force-unlock` fails due to backend issues, you can directly delete the lock record from DynamoDB using the AWS CLI. After unlocking, always run `terraform plan` before the next apply to check the state is consistent."

Topic 93: Silent Drift — Plan Shows Clean But Infra Is Different

---

🔵 What It Is (Simple Terms)

The most dangerous form of drift — `terraform plan` shows "No changes. Infrastructure is up-to-date." but someone has made manual changes to your infrastructure. How is this possible?

🔵 Root Causes

Cause 1: `ignore_changes` is Hiding Drift

```
resource "aws_instance" "web" {
  instance_type = "t3.medium"

  lifecycle {
```

```
    ignore_changes = [instance_type, tags]   # ← drift on these is
silently ignored
  }
}
# Someone changed instance_type to "t3.large" manually
# terraform plan says: No changes ← WRONG, but by design
```

Cause 2: `-refresh=false` Was Used

```
# Plan was run with refresh disabled
terraform plan -refresh=false
# This reads state but DOESN'T call AWS APIs to check reality
# State says t3.medium → plan says no changes
# Reality is t3.large → plan doesn't know
```

Cause 3: Attribute Not Tracked in State

```
# Some provider attributes are not tracked in state
# particularly with older providers or partial schema implementations
# Manual changes to untracked attributes are invisible to Terraform
```

Cause 4: Out-of-Band Resource Creation

```
# Resources created outside Terraform are completely invisible
# Terraform only plans for resources it knows about
# A manually created S3 bucket won't show in plan at all
```

Cause 5: Computed Attribute Changed

```
# Some attributes are marked "computed" — Terraform doesn't manage them
# Changes to computed attributes don't show in plan
resource "aws_s3_bucket" "data" {
  bucket = "myapp-data"
  # region is computed — if somehow it changed, plan wouldn't show it
}
```

🔵 Detection Tools

```
# 1. Full refresh plan (detects attribute drift on managed resources)
terraform plan -refresh=true
```

```
# 2. refresh-only apply (shows all attribute drift)
terraform apply -refresh-only

# 3. AWS Config — detects ALL changes including non-Terraform resources
# 4. CloudTrail — audit log of all API changes
# 5. AWS Security Hub — compliance-based drift detection
# 6. Prowler / ScoutSuite — security-focused drift detection
# 7. Driftctl — open source tool specifically for Terraform drift
detection
```

🔵 Short Interview Answer

> "Silent drift — plan shows clean but infra differs — has four main causes: `ignore_changes` in the lifecycle block deliberately ignoring specific attributes, `-refresh=false` being used which skips API calls entirely, untracked computed attributes that providers don't include in state diffs, and out-of-band resources created outside Terraform that are completely invisible to it. For managed resource drift, `terraform plan -refresh=true` or `terraform apply -refresh-only` catches it. For unmanaged resources, you need external tools like AWS Config, CloudTrail, or Driftctl which inventory the cloud and compare against your Terraform state."

Topic 94: Perpetual Diff Debugging

🔵 What It Is (Simple Terms)

Every time you run `terraform plan`, a resource shows a change — even though you haven't modified anything. You apply it, the change completes, and the next plan shows the same change again. Infinite loop of diffs.

🔵 Root Causes and Fixes

Cause 1: Provider Normalizes Values Differently

```
# You set:
resource "aws_s3_bucket_policy" "main" {
  policy = jsonencode({
    Version = "2012-10-17"
    Statement = [{
      Effect   = "Allow"
      Action   = "s3:GetObject"
      Resource = "arn:aws:s3:::mybucket/*"
    }]
  })
}
```

```
# AWS stores it as:
# {"Version":"2012-10-17","Statement":[{"Effect":"Allow",...}]}
# Your config produces different whitespace/ordering each plan
# Plan shows: ~ policy = ... (JSON looks different but is semantically
identical)

# Fix: Use aws_iam_policy_document data source instead
data "aws_iam_policy_document" "main" {
  statement {
    effect    = "Allow"
    actions   = ["s3:GetObject"]
    resources = ["arn:aws:s3:::mybucket/*"]
  }
}

resource "aws_s3_bucket_policy" "main" {
  policy = data.aws_iam_policy_document.main.json  # stable output
}
```

Cause 2: `timestamp()` in Resource Config

```
# ❌ timestamp() changes on every plan
resource "aws_s3_bucket" "data" {
  tags = {
    LastUpdated = timestamp()   # changes every plan → perpetual diff
  }
}

# ✅ Fix: ignore_changes on the timestamp tag
resource "aws_s3_bucket" "data" {
  tags = {
    LastUpdated = timestamp()
  }
  lifecycle {
    ignore_changes = [tags["LastUpdated"]]
  }
}
```

Cause 3: Provider Reads Attribute Differently Than Written

```
# AWS may store/return values in a normalized form
# e.g., IAM policies have Statement reordering
# e.g., Security group descriptions have trailing spaces stripped
# e.g., KMS key policy has Principal formatting differences

# Fix: Use ignore_changes for the perpetually-diffing attribute
lifecycle {
  ignore_changes = [policy]
}
```

```
# Or: Match the format AWS returns exactly
```

Cause 4: External System Modifying the Resource

```
# An external system (auto-scaler, config management, another Terraform
stack)
# is modifying the resource after Terraform applies

# Fix: Identify what's modifying it (CloudTrail)
# Then either:
# a) Stop the external modification
# b) Use ignore_changes for the affected attribute
# c) Remove the resource from Terraform management
```

Cause 5: List/Set Ordering

```
# Provider stores as set (unordered), you provide as list (ordered)
# Terraform sees ordering difference every plan

resource "aws_security_group" "web" {
  # ❌ List order may not match what AWS returns
  ingress {
    cidr_blocks = ["10.0.0.0/8", "172.16.0.0/12"]
  }
}

# Fix: toset() or sort the list to match AWS's canonical ordering
```

🔵 Debugging Process

```
# Step 1: Identify which attribute is perpetually diffing
terraform plan 2>&1 | grep "~\|+\|-"

# Step 2: Enable DEBUG logging to see raw API responses
export TF_LOG=DEBUG
terraform plan 2>&1 | grep -A5 "perpetually_diffing_attribute"

# Step 3: Check what AWS actually stores
aws [service] describe-[resource] --[id] [value]
# Compare: what you set vs what AWS returns

# Step 4: Check provider GitHub issues
# Search: "perpetual diff <resource_type> <attribute>"
# Often a known provider bug with a workaround
```

```
# Step 5: Apply the fix
# Option A: ignore_changes = [attribute]
# Option B: match the exact format AWS expects
# Option C: use a data source or provider function to normalize
```

🔵 Short Interview Answer

> "Perpetual diffs happen when Terraform always sees a difference between config and state even after applying. The most common causes are: JSON policy formatting where AWS normalizes the JSON differently than your config produces it (fix: use `aws_iam_policy_document`), `timestamp()` in resource attributes which changes every plan (fix: `ignore_changes`), provider behavior where it reads a value back in a different format than you wrote it (fix: match AWS's canonical format or `ignore_changes`), and list vs set ordering mismatches. Debug by enabling `TF_LOG=DEBUG` to see raw API responses and compare them to what your config produces."

Topic 95: Plan Passes But Apply Fails

🔵 What It Is (Simple Terms)

`terraform plan` completes successfully, you approve it, run `terraform apply` — and it fails. The plan lied, or the world changed between plan and apply.

🔵 Root Causes

Cause 1: Infrastructure Changed Between Plan and Apply

```
10:00 AM: terraform plan — looks good, 3 resources to create
10:05 AM: Another team deletes the subnet your EC2 instance would go into
10:06 AM: terraform apply — fails: "subnet-abc123 not found"

Time gap between plan and apply is the enemy.
In CI/CD: minimize time between plan and apply
In manual workflows: re-plan if significant time has passed
```

Cause 2: IAM Permissions Insufficient at Apply Time

```
# Plan only reads (LIST, DESCRIBE, GET)
# Apply writes (CREATE, UPDATE, DELETE)

# A role may have read permissions but not write permissions
```

```
# Plan succeeds (read-only) → Apply fails (write forbidden)

# Error example:
# Error: creating EC2 Instance: UnauthorizedOperation:
# You are not authorized to perform this operation.

# Fix: Ensure Terraform execution role has full permissions
# for all resources it manages, not just read permissions
```

Cause 3: Eventual Consistency — Resource Not Ready

```
# IAM role created → immediately try to attach policy → "NoSuchEntity"
# IAM propagation takes seconds to minutes

# Error:
# Error: attaching IAM Policy to IAM Role: NoSuchEntityException:
# Role prod-lambda-role does not exist

# Fix: depends_on on downstream resources
# Or: time_sleep resource
resource "time_sleep" "iam_propagation" {
  depends_on      = [aws_iam_role.lambda]
  create_duration = "10s"
}
```

Cause 4: Resource Limit/Quota Reached

```
# Plan doesn't check service quotas
# Apply hits the limit and fails

# Error:
# Error: creating EC2 Instance: InstanceLimitExceeded:
# You have reached your maximum limit of 32 instances for this region

# Fix: Request quota increases before applying large configs
# Use AWS Service Quotas to check limits before apply
```

Cause 5: Computed Value Conflict

```
# Plan shows (known after apply) for a value
# At apply time, the computed value conflicts with another resource

# Example: two resources trying to use the same auto-assigned port
# Plan can't detect this — computed values are unknown at plan time
```

Cause 6: API Throttling During Apply

```
# Large applies make many API calls simultaneously
# AWS throttles: "RequestLimitExceeded" or "Throttling"

# Fix:
terraform apply -parallelism=5   # reduce concurrent API calls
# Default parallelism is 10 — reduce for throttling-prone accounts
```

🔵 Recovery After Partial Apply Failure

```
# After a failed apply, check state:
terraform state list              # what was created before failure?
terraform plan                    # what still needs to be created?

# Usually: just re-run terraform apply
# Terraform will skip already-created resources and retry failed ones

# If state is inconsistent after failure:
terraform refresh                 # sync state with reality
terraform plan                    # assess full picture
terraform apply                   # converge
```

🔵 Short Interview Answer

> "Plan passes but apply fails for several reasons. Most common: infrastructure changed between plan and apply — another team deleted a resource your config depends on. IAM permission gaps — plan only needs read permissions but apply needs write, so it passes plan and fails apply. Eventual consistency — IAM roles created at plan time may not have propagated when the policy attachment runs at apply. Resource quota limits — plan doesn't check service quotas. And API throttling during large applies. Best practice is to minimize the time between plan and apply in CI/CD, use saved plan files so you apply exactly what was planned, and add retry logic for throttling errors."

Topic 96: `(known after apply)` Cascading

🔵 What It Is (Simple Terms)

`(known after apply)` means Terraform can't determine a value at plan time — it'll only be known after a resource is created. The problem is when this unknown value is used by another resource, which makes that resource's plan also unknown, which cascades to a third resource, and so on — making your plan output full of unknowns.

🔵 How It Cascades

```
aws_instance.web:
  id = (known after apply)         ← computed, only known after creation

aws_eip_association.web:
  instance_id = aws_instance.web.id   ← references the unknown ID
  instance_id = (known after apply)   ← now this is also unknown

aws_route53_record.web:
  records = [aws_eip_association.web.public_ip]  ← references unknown
  records = [(known after apply)]                ← now this is also
unknown

aws_lb_target_group_attachment.web:
  target_id = aws_instance.web.id   ← unknown
  target_id = (known after apply)   ← cascades again
```

🔵 When It Becomes a Problem

```
# Mostly benign — this is normal Terraform behavior for new resources
# Becomes a PROBLEM when:

# 1. A count or for_each uses an unknown value
resource "aws_route53_record" "web" {
  count   = length(aws_instance.web[*].id)  # unknown count = can't plan!
  # Error: The "count" value depends on resource attributes that cannot
  # be determined until apply, so Terraform cannot predict how many
  # instances will be created.
}

# 2. Provider validation fails on unknown values
# Some providers validate attribute values at plan time
# If the value is unknown, validation is skipped
# The error only surfaces at apply — see Topic 95

# 3. Conditional expressions with unknown values
locals {
  use_new_instance = aws_instance.web.id != null   # unknown at plan time
  # Terraform can't evaluate the conditional → unknown propagates
}
```

🔵 Fixes for `count` with Unknown Values

```
# ❌ Problem: count depends on unknown value
resource "aws_security_group_rule" "web" {
  count       = length(aws_instance.web)    # unknown if instances aren't
created yet
  type        = "ingress"
  from_port   = 80
  to_port     = 80
  protocol    = "tcp"
  cidr_blocks = [aws_instance.web[count.index].private_ip]
}

# ✅ Fix Option 1: Use a known value for count
resource "aws_security_group_rule" "web" {
  count       = var.instance_count          # known at plan time
  cidr_blocks = [aws_instance.web[count.index].private_ip]
}

# ✅ Fix Option 2: Use for_each with a known map
resource "aws_security_group_rule" "web" {
  for_each    = var.instances               # known map of instance
configs
  cidr_blocks = [aws_instance.web[each.key].private_ip]
}

# ✅ Fix Option 3: Split into two applies
# Apply 1: Create instances (now their IDs are known)
# Apply 2: Create rules referencing known IDs
```

🔵 Short Interview Answer

> "`(known after apply)` cascades when resource A's computed attribute is used by resource B's configuration, making B's value unknown too — and this propagates to C, D, and so on. For most attributes this is fine — Terraform handles it gracefully. It becomes a real problem when an unknown value is used in `count` or `for_each`, because Terraform can't determine how many resources to create at plan time and errors out with 'The count value depends on resource attributes that cannot be determined until apply.' The fix is to ensure count and for_each always use known values — use input variables or local values rather than computed resource attributes."

Topic 97: Safe Resource Renaming

🔵 What It Is (Simple Terms)

You want to rename a resource in your Terraform config — change `aws_instance.web` to `aws_instance.web_server`. Without proper handling, Terraform sees this as "destroy old resource, create new one" — causing downtime and data loss.

🔵 The Problem

```
# Before:
resource "aws_instance" "web" {
  ami           = "ami-abc"
  instance_type = "t3.medium"
}

# After rename:
resource "aws_instance" "web_server" {   # renamed
  ami           = "ami-abc"
  instance_type = "t3.medium"
}

# terraform plan shows:
# - aws_instance.web will be destroyed    ← DESTROYS the instance
# + aws_instance.web_server will be created  ← RECREATES it
# Plan: 1 to add, 0 to change, 1 to destroy
```

🔵 The Fix: `moved` Block (Terraform 1.1+)

```
# Step 1: Rename the resource in config
resource "aws_instance" "web_server" {   # new name
  ami           = "ami-abc"
  instance_type = "t3.medium"
}

# Step 2: Add moved block to tell Terraform about the rename
moved {
  from = aws_instance.web          # old address
  to   = aws_instance.web_server   # new address
}

# terraform plan now shows:
# Terraform will move resource aws_instance.web to aws_instance.web_server
# Plan: 0 to add, 0 to change, 0 to destroy  ← ZERO CHANGES!
```

🔵 `moved` Block for Module Moves

```
# Moving a resource INTO a module
moved {
  from = aws_instance.web
  to   = module.app.aws_instance.web
}
```

```
# Moving a resource between modules
moved {
  from = module.app_v1.aws_instance.web
  to   = module.app_v2.aws_instance.web
}

# Renaming a module itself
moved {
  from = module.database
  to   = module.rds
}
```

🔵 The Old Way: `terraform state mv`

```
# Before moved block existed (still valid, but less elegant)
terraform state mv aws_instance.web aws_instance.web_server

# Then rename in config
# Run plan — should show 0 changes
```

🔵 Cleaning Up `moved` Blocks

```
# After everyone has applied the moved block:
# It's safe to remove it — state already reflects the new name
# But keep it for a period to allow all team members to apply

# Best practice: keep moved blocks for 1-2 releases
# Then remove them with a comment in the PR:
# "Remove moved block — all environments have applied this change"
```

🔵 Short Interview Answer

> "Renaming a resource in Terraform config without handling it causes Terraform to destroy the old resource and create a new one — dangerous for stateful resources. The modern fix is the `moved` block (Terraform 1.1+): rename the resource in config, add `moved { from = old_name, to = new_name }`, and Terraform updates only the state file without touching real infrastructure. Plan shows zero destroys/creates. The alternative is `terraform state mv old_name new_name` which achieves the same result but requires a manual CLI step rather than being codified in config. Always keep `moved` blocks for a few releases so all team members and environments can apply the state rename."

Topic 98: `count` to `for_each` Live Migration

🔵 What It Is (Simple Terms)

You have resources created with `count` and need to migrate them to `for_each` — without destroying and recreating them. This is one of the trickiest Terraform operations because `count` and `for_each` use completely different state addressing.

🔵 Why It's Destructive By Default

```
# Current state (count-based):
# aws_instance.web[0]  → i-aaa
# aws_instance.web[1]  → i-bbb
# aws_instance.web[2]  → i-ccc

# After switching to for_each:
resource "aws_instance" "web" {
  for_each = toset(["web-a", "web-b", "web-c"])
  # ...
}
# New state addresses:
# aws_instance.web["web-a"]  → needs to be created
# aws_instance.web["web-b"]  → needs to be created
# aws_instance.web["web-c"]  → needs to be created

# Plan shows:
# + aws_instance.web["web-a"] will be created
# + aws_instance.web["web-b"] will be created
# + aws_instance.web["web-c"] will be created
# - aws_instance.web[0] will be destroyed
# - aws_instance.web[1] will be destroyed
# - aws_instance.web[2] will be destroyed
# Plan: 3 to add, 0 to change, 3 to destroy  ← DESTROYS ALL INSTANCES
```

🔵 The Safe Migration Path

```
# Step 1: Map old index addresses to new key addresses
# aws_instance.web[0] → aws_instance.web["web-a"]
# aws_instance.web[1] → aws_instance.web["web-b"]
# aws_instance.web[2] → aws_instance.web["web-c"]

# Step 2: Use terraform state mv to rename each address
terraform state mv 'aws_instance.web[0]' 'aws_instance.web["web-a"]'
terraform state mv 'aws_instance.web[1]' 'aws_instance.web["web-b"]'
terraform state mv 'aws_instance.web[2]' 'aws_instance.web["web-c"]'
```

```
# Step 3: Update config to use for_each
resource "aws_instance" "web" {
  for_each      = toset(["web-a", "web-b", "web-c"])
  ami           = data.aws_ami.ubuntu.id
  instance_type = "t3.medium"
  tags          = { Name = each.key }
}

# Step 4: Run plan — should show 0 changes
terraform plan
# Expected: No changes. Infrastructure is up-to-date.
```

🔵 Using `moved` Block for the Migration

```
# Terraform 1.1+ — cleaner approach with moved blocks

# New config:
resource "aws_instance" "web" {
  for_each = toset(["web-a", "web-b", "web-c"])
  # ...
}

# moved blocks for each instance
moved {
  from = aws_instance.web[0]
  to   = aws_instance.web["web-a"]
}

moved {
  from = aws_instance.web[1]
  to   = aws_instance.web["web-b"]
}

moved {
  from = aws_instance.web[2]
  to   = aws_instance.web["web-c"]
}
```

🔵 Short Interview Answer

> "Migrating from `count` to `for_each` is destructive by default because they use different state address formats — `resource[0]` vs `resource[\"key\"]`. Terraform sees the old indexed resources as deleted and the new keyed resources as new. The safe path is to use `terraform state mv` to rename each state entry from its index address to its new key address before updating the config — or equivalently, use `moved` blocks in Terraform 1.1+. After the state rename, update the config to use `for_each` and run `terraform plan` — it should show zero changes. The hard part is

> mapping `[0]`, `[1]`, `[2]` to meaningful keys — make sure the mapping is correct or you'll swap which real resource gets which state entry."

Topic 99: Root Module to Child Module Refactoring

🔵 What It Is (Simple Terms)

You have a large root module with 50+ resources all in one place. You want to reorganize them into logical child modules — without destroying and recreating anything.

🔵 Why It's Risky

```
Root module resources:
  aws_vpc.main
  aws_subnet.private[0]
  aws_subnet.private[1]
  aws_instance.web
  aws_instance.api
  aws_db_instance.main

Moving to module "networking":
  Old address: aws_vpc.main
  New address: module.networking.aws_vpc.main   ← completely different
  Terraform plan: destroy old, create new ← WRONG
```

🔵 The `moved` Block Solution

```
# Step 1: Create the module structure
# modules/networking/main.tf
resource "aws_vpc" "main" { ... }
resource "aws_subnet" "private" { ... }

# Step 2: Call the module from root
module "networking" {
  source = "./modules/networking"
  # pass variables
}

# Step 3: Add moved blocks in ROOT module
moved {
  from = aws_vpc.main
  to   = module.networking.aws_vpc.main
}
```

```
moved {
  from = aws_subnet.private[0]
  to   = module.networking.aws_subnet.private[0]
}

moved {
  from = aws_subnet.private[1]
  to   = module.networking.aws_subnet.private[1]
}

# Step 4: Run plan — should show 0 destroys
terraform plan
# Shows only: "Terraform will move N resource(s)"
# Plan: 0 to add, 0 to change, 0 to destroy
```

🔵 Migration Checklist

```
Pre-migration:
  ✅ Back up state file
  ✅ Create module with identical resource configs (copy-paste first)
  ✅ Ensure module variable interface matches what root passes
  ✅ Write all moved blocks before running any plan

During migration:
  ✅ Run terraform plan — verify 0 destroy
  ✅ Verify moved block addresses are exact (copy from state list)
  ✅ Apply the migration

Post-migration:
  ✅ Run plan again — should be 0 changes
  ✅ Keep moved blocks for all environments to apply
  ✅ Remove original resource blocks from root (they're now in module)
  ✅ Update all references to use module.name.output_name
```

🔵 Short Interview Answer

> "Moving resources from root module to child modules changes their state addresses — from `aws_vpc.main` to `module.networking.aws_vpc.main`. Without `moved` blocks, Terraform destroys the old addresses and creates new ones. The safe approach: create the module, call it from root, and add a `moved` block for every resource being moved, mapping the old root address to the new module address. Plan should show zero destroys. The most important step is backing up state first and verifying the plan shows only `moved` operations, not creates or destroys."

Topic 100: Third-Party Module Breaking Change Handling

🔵 What It Is (Simple Terms)

You're using a community or partner Terraform module. They publish a new version with breaking changes — renamed variables, restructured resources, removed outputs. Your next apply could destroy production infrastructure.

🔵 How It Happens

```
# Your config — pinned to a module version
module "vpc" {
  source  = "terraform-aws-modules/vpc/aws"
  version = "~> 4.0"      # allows 4.x updates
}

# Module publishes v4.5.0 with breaking change:
# - Renames private_subnets output to private_subnet_ids
# - Changes internal resource address for subnets

# terraform init -upgrade picks up v4.5.0
# terraform plan shows: DESTROY AND RECREATE ALL SUBNETS
```

🔵 Prevention

```
# Pin to exact version — never use ~> for third-party modules in prod
module "vpc" {
  source  = "terraform-aws-modules/vpc/aws"
  version = "= 4.4.0"   # exact pin
}

# Test upgrades in dev first:
# 1. Upgrade in dev environment
# 2. Run plan — check for unexpected destroys
# 3. Read the module's CHANGELOG for breaking changes
# 4. Apply and verify
# 5. Promote to staging, then prod
```

🔵 Recovery Steps

```
# Step 1: Revert the module version immediately
# Change version back to last known good version
# terraform init to downgrade

# Step 2: Read the module changelog
# https://github.com/terraform-aws-modules/terraform-aws-
vpc/blob/master/CHANGELOG.md
# Understand exactly what changed

# Step 3: Plan the upgrade carefully
# Check if moved blocks are provided in the new module version
# Good module authors provide moved blocks for resource renames

# Step 4: If module provides no migration path
# Use terraform state mv to manually map old → new resource addresses
# Test thoroughly in non-prod first

# Step 5: Apply incrementally
# Use -target to apply non-destructive changes first
# Then address breaking changes one by one
```

🔵 Short Interview Answer

> "Third-party module breaking changes can trigger unexpected destroys when you upgrade. Prevention: pin to exact module versions in production (`= 4.4.0`, not `~> 4.0`), never use `terraform init -upgrade` in production without planning first, and read changelogs before upgrading. Recovery: immediately revert the version pin to the last good version, read the changelog to understand what changed structurally, and check if the new module version includes `moved` blocks (good module authors do this). If not, manually use `terraform state mv` to map old resource addresses to new ones. Always test module upgrades in dev/staging with a full plan review before production."

Topic 101: Accidental Secret Commit Recovery

🔵 What It Is (Simple Terms)

A developer committed a `terraform.tfvars` file containing passwords, API keys, or credentials to Git. The secret is now in version history — even deleting the file doesn't remove it from history.

🔵 Immediate Response (First 15 Minutes)

```
# Step 1: IMMEDIATELY rotate/revoke the secret
# Don't wait to clean Git — assume it's compromised NOW
```

```
aws iam delete-access-key --access-key-id AKIAIOSFODNN7EXAMPLE
# Or rotate the DB password, revoke the API key, etc.

# Step 2: Check if the repo is public or private
# Public: assume secret is already scraped by bots (they monitor GitHub in
real time)
# Private: assess who has access — check audit logs

# Step 3: Alert security team
# Incident response procedure
```

🔵 Git History Purge

```
# Using BFG Repo-Cleaner (faster than git filter-branch)
# Step 1: Create a file listing secrets to remove
echo "super-secret-password" > passwords.txt

# Step 2: Run BFG
bfg --replace-text passwords.txt myrepo.git

# Step 3: Clean and force push
cd myrepo
git reflog expire --expire=now --all
git gc --prune=now --aggressive
git push --force --all
git push --force --tags

# Alternative: git filter-branch (slower but built-in)
git filter-branch --force --index-filter \
  "git rm --cached --ignore-unmatch terraform.tfvars" \
  --prune-empty --tag-name-filter cat -- --all

# Step 4: Notify all team members to re-clone
# Cached versions on developer machines still have history
```

🔵 Prevention

```
# .gitignore — ALWAYS have this in every Terraform repo
*.tfvars              # ignore all tfvars
!example.tfvars       # except example/template files
*.tfvars.json
terraform.tfstate
terraform.tfstate.backup
.terraform/
```

```
# Pre-commit hooks — catch secrets before they're committed
# Install detect-secrets or git-secrets
pip install detect-secrets
detect-secrets scan > .secrets.baseline
detect-secrets audit .secrets.baseline

# Or use pre-commit framework
# .pre-commit-config.yaml:
repos:
  - repo: https://github.com/Yelp/detect-secrets
    rev: v1.4.0
    hooks:
      - id: detect-secrets
```

🔵 Short Interview Answer

> "First priority when a secret is committed to Git: immediately rotate and revoke the compromised credential — don't wait for Git cleanup, assume it's compromised. Then purge it from Git history using BFG Repo-Cleaner or `git filter-branch`, followed by a force push. Notify all team members to re-clone since cached local repos still have history. Prevention: `.gitignore` all `*.tfvars` files except example templates, use `TF_VAR_` environment variables for secrets injected by CI/CD, and add pre-commit hooks with tools like `detect-secrets` to catch secrets before they're committed."

Topic 102: Sensitive Value Leak Tracing

🔵 What It Is (Simple Terms)

A sensitive value — database password, API key, private key — is appearing in plan output, apply logs, or Terraform Cloud run logs even though you thought it was protected. Tracing where the leak is happening.

🔵 Common Leak Paths

```
Path 1: Variable not marked sensitive
  variable "db_password" { type = string }  # ← missing sensitive = true
  → Shows in plain text in plan output

Path 2: Sensitive variable used in non-sensitive local
  locals {
    conn_string = "postgres://${var.db_password}@${var.db_host}"
    # conn_string inherits sensitivity in TF 0.15+
    # BUT if you explicitly use it in a non-sensitive context it may leak
  }
```

```
Path 3: Sensitive output not marked sensitive
  output "db_url" {
    value = local.conn_string   # TF 0.15+ errors here — you must mark
sensitive
  }

Path 4: `local-exec` provisioner logging
  provisioner "local-exec" {
    command = "echo ${var.db_password} | configure-db.sh"  # leaked in
apply log!
  }

Path 5: Terraform Cloud / CI/CD log streaming
  # Even redacted values can appear in API response logs
  # if debug logging is enabled (TF_LOG=DEBUG)
```

🔵 Tracing the Leak

```
# Step 1: Find where the sensitive variable is used
grep -r "var.db_password" .
grep -r "local.connection_string" .

# Step 2: Check if all paths have sensitive = true or inherit it
# In Terraform 0.15+, using a sensitive value in an output without
# marking it sensitive causes an error — this catches most leaks

# Step 3: Check provisioners
grep -r "local-exec\|remote-exec" .
# Any provisioner command containing sensitive values will leak

# Step 4: Check TF_LOG level in CI/CD
# TF_LOG=DEBUG logs raw HTTP requests including request bodies
# → passwords in API calls can appear in debug logs

# Step 5: Audit Terraform Cloud run logs
# Even with sensitive = true, some older versions leaked in specific
contexts
```

🔵 Fixes

```
# Fix 1: Mark all sensitive variables
variable "db_password" {
  type      = string
  sensitive = true   # ← add this
}
```

```
# Fix 2: Mark outputs that contain sensitive values
output "db_connection" {
  value     = local.connection_string
  sensitive = true   # ← required when value contains sensitive data
}

# Fix 3: Never put sensitive values in provisioner commands
# ❌ Wrong:
provisioner "local-exec" {
  command = "aws configure set aws_secret_access_key ${var.secret_key}"
}

# ✅ Better: write to a file, use env vars
provisioner "local-exec" {
  command     = "configure.sh"
  environment = {
    SECRET_KEY = var.secret_key   # still leaks in DEBUG logs but better
  }
}

# Fix 4: Use Vault/SSM at runtime — don't pass secrets through Terraform
at all
```

🔵 Short Interview Answer

> "Sensitive value leaks in Terraform happen through several paths: forgetting `sensitive = true` on the variable, using sensitive values in `local-exec` provisioner commands which appear in apply logs, marking a local value as sensitive but then using it in a non-sensitive output (Terraform 0.15+ now errors on this), and `TF_LOG=DEBUG` which logs raw HTTP request bodies including API calls that contain the sensitive value. Trace by grepping for all uses of the sensitive variable and verifying each usage is in a sensitive-marked context. The real fix is to not pass secrets through Terraform at all — fetch them from Vault or SSM in the application at runtime."

Topic 103: Retroactive State Encryption

🔵 What It Is (Simple Terms)

Your Terraform state is stored in S3 without encryption. Security audit flags it. You need to enable encryption without losing or corrupting the state file.

🔵 The Risk

```
State file contains:
  - Resource IDs and ARNs
```

```
  - All resource attributes
  - Sensitive variable values (passwords, keys) in plaintext

Unencrypted S3 bucket = anyone with S3 access can read all of this
```

🔵 Enabling S3 SSE Encryption (Non-Disruptive)

```
# Step 1: Enable default encryption on the S3 bucket
# This encrypts NEW objects — existing objects are NOT retroactively
encrypted
aws s3api put-bucket-encryption \
  --bucket mycompany-terraform-state \
  --server-side-encryption-configuration '{
    "Rules": [{
      "ApplyServerSideEncryptionByDefault": {
        "SSEAlgorithm": "aws:kms",
        "KMSMasterKeyID": "arn:aws:kms:us-east-1:123456789012:key/abc123"
      }
    }]
  }'

# Step 2: Re-upload existing state files to encrypt them
# Method: copy object to itself with encryption header
aws s3 cp \
  s3://mycompany-terraform-state/prod/terraform.tfstate \
  s3://mycompany-terraform-state/prod/terraform.tfstate \
  --sse aws:kms \
  --sse-kms-key-id arn:aws:kms:us-east-1:123456789012:key/abc123

# This overwrites the object in-place with encryption applied
# The state content is unchanged — just encrypted at rest now
```

🔵 Update Terraform Backend Config

```
# After enabling encryption, update backend config
terraform {
  backend "s3" {
    bucket  = "mycompany-terraform-state"
    key     = "prod/terraform.tfstate"
    region  = "us-east-1"
    encrypt = true               # ← add this flag
    kms_key_id = "arn:aws:kms:us-east-1:123456789012:key/abc123"
  }
}

# Run terraform init to reconfigure the backend
terraform init -reconfigure
```

🔵 Migrating to Fully Encrypted Backend

```
# If moving from unencrypted to new encrypted bucket:

# Step 1: Back up current state
terraform state pull > backup.tfstate

# Step 2: Update backend config to new encrypted bucket
# (update .tf files)

# Step 3: Migrate state
terraform init -migrate-state
# Terraform asks: "Do you want to migrate your state?"
# Type "yes"

# Step 4: Verify
terraform plan   # should show 0 changes
```

🔵 Short Interview Answer

> "Enabling S3 encryption retroactively is non-disruptive. First, enable default encryption on the S3 bucket — this encrypts new writes but existing objects remain unencrypted. Then re-encrypt existing state files by using `aws s3 cp` to copy each object to itself with the `--sse aws:kms` flag — this overwrites the object with encryption applied without changing the content. Finally, add `encrypt = true` and optionally a KMS key ID to the Terraform backend config and run `terraform init -reconfigure`. The state content is unchanged — just encrypted at rest. No state migration or downtime required."

Topic 104: Slow Plan Debugging

🔵 What It Is (Simple Terms)

`terraform plan` is taking 15+ minutes. CI/CD pipelines time out. Engineers avoid running plans. This is a real productivity and safety problem — people skip plans to avoid the wait.

🔵 Root Causes

Cause 1: Large State File with Many Resources

```
Each resource = one or more AWS API calls during refresh
500 resources × 0.5s per API call = 250 seconds minimum
1000 resources = 8+ minutes just for refresh
```

Cause 2: API Throttling

```
# Terraform makes many parallel API calls (default: 10 concurrent)
# AWS throttles certain APIs:
# EC2 DescribeInstances: 100 calls/second
# IAM GetRole: 20 calls/second (lower limit!)
# Route53 ListResourceRecordSets: 5 calls/second (very low!)

# Throttled calls → retry with backoff → each retry takes seconds
# Many retries → plan takes minutes

# Identify throttling in logs:
export TF_LOG=DEBUG
terraform plan 2>&1 | grep -i "throttl\|RequestLimitExceeded\|rate"
```

Cause 3: Slow Provider Operations

```
# Some data sources are inherently slow:
data "aws_ami" "ubuntu" {
  most_recent = true
  # Searches all AMIs → can be slow with broad filters
}

# data sources that call many APIs:
data "aws_instances" "all" {}  # describes ALL instances → very slow
```

Cause 4: Many Data Sources

```
# Data sources are evaluated during plan
# 50 data sources × 1s each = 50s just for data sources
# data.aws_caller_identity is fast
# data.aws_ami with broad filters is slow
```

🔵 Debugging Which Resources Are Slow

```
# Enable timing in logs
export TF_LOG=DEBUG
export TF_LOG_PATH=plan-debug.log
```

```
time terraform plan

# Analyze which resources take longest
grep "provider.aws:" plan-debug.log | grep -E "elapsed|duration"

# Check which API calls are being made
grep "GET\|POST\|PUT\|DELETE" plan-debug.log | \
  awk '{print $NF}' | sort | uniq -c | sort -rn
```

🔵 Fixes

```
# Fix 1: Split large state files (see Topic 105)
# Reduce resources per state → fewer API calls per plan

# Fix 2: Reduce parallelism to avoid throttling
terraform plan -parallelism=5   # fewer concurrent API calls

# Fix 3: Use -refresh=false for known-stable environments
terraform plan -refresh=false   # skips API calls entirely — use carefully

# Fix 4: Narrow data source filters
data "aws_ami" "ubuntu" {
  most_recent = true
  owners      = ["099720109477"]   # ← specify owner, not just filters
  filter {
    name   = "name"
    values = ["ubuntu/images/hvm-ssd/ubuntu-jammy-22.04-amd64-server-
20240101"]
    # ← exact name, not wildcard, is much faster
  }
}

# Fix 5: Cache data source results in locals (avoid duplicate calls)
# Fix 6: Upgrade provider — newer providers often have better API
efficiency
# Fix 7: Use provider-level retry/timeout configs for throttled APIs
```

🔵 Short Interview Answer

> "Slow plans have four main causes: large state files where refresh calls APIs for every resource (500 resources can mean 5+ minutes), API throttling where AWS rate-limits calls and Terraform retries with backoff, slow data sources with broad filters, and too many data sources. Debug with `TF_LOG=DEBUG` to see which API calls are slow or being throttled. Fixes: split large state files, reduce `-parallelism` to avoid throttling, narrow data source filters to avoid broad searches, and use `-refresh=false` carefully in known-stable environments to skip the refresh phase entirely."

Topic 105: Large State File Management

🔵 What It Is (Simple Terms)

A Terraform state file that has grown to manage hundreds or thousands of resources — causing slow plans, risky applies (one mistake affects everything), and difficult team collaboration.

🔵 Warning Signs

```
⚠ terraform plan takes > 5 minutes
⚠ State file is > 10MB
⚠ > 200 resources in one state file
⚠ Multiple teams need to apply to the same config
⚠ A single failed apply blocks all infrastructure changes
⚠ Engineers afraid to make changes due to blast radius
```

🔵 Splitting Strategy

```
Before (monolithic):
  state/prod/terraform.tfstate  ← all 500 resources

After (split by layer/domain):
  state/prod/networking/terraform.tfstate      ← VPC, subnets, routes (50
resources)
  state/prod/security/terraform.tfstate        ← IAM, security groups (80
resources)
  state/prod/data/terraform.tfstate            ← RDS, ElastiCache (30
resources)
  state/prod/compute/terraform.tfstate         ← EKS, EC2, ASG (150
resources)
  state/prod/applications/terraform.tfstate    ← app-level resources (190
resources)
```

🔵 Splitting Principles

```
Split by:
  1. Team ownership — each team owns their state
  2. Rate of change — stable infra (VPC) separate from volatile infra
(apps)
  3. Blast radius — breaking one state doesn't affect others
  4. Dependency direction — networking → compute → application (one
direction)
```

```
Don't split by:
  1. Convenience — splitting creates coordination overhead
  2. Too granular — 1-5 resources per state is too small
  3. Circular dependencies — A needs B and B needs A = problem
```

🔵 The Migration Process

```
# Step 1: Identify resource groupings
terraform state list | sort > all-resources.txt
# Manually group into logical domains

# Step 2: For each target state file:
# a. Create new Terraform config directory
# b. Move resources using terraform state mv (cross-state)
# c. Add resource configs to new directory
# d. Verify plan shows 0 changes

# Step 3: Cross-stack references
# Old: directly reference resource attributes
# New: use outputs + terraform_remote_state or SSM parameters

# Step 4: Remove resources from original state
# After verification, remove from source config + state

# Step 5: Update CI/CD
# Add separate pipeline stages for each state
```

🔵 Short Interview Answer

> "Large state files cause slow plans, high blast radius, and team friction. The solution is splitting by domain — networking, security, data, compute, applications — with each domain in its own state file managed by a separate Terraform config. Split along team ownership boundaries and dependency direction (networking → compute → apps, never circular). Migration uses `terraform state mv` to move resources between state files, with `moved` blocks in configs to handle address changes. Cross-stack references switch from direct attribute access to outputs via `terraform_remote_state` or SSM Parameter Store. The goal is each state having 50-200 resources — small enough for fast plans, large enough to avoid coordination overhead."

Topic 106: Parallelism Tuning

🔵 What It Is (Simple Terms)

Terraform applies changes in parallel — by default 10 resources at a time. Tuning this number balances speed against API throttling and resource ordering issues.

🔵 The `-parallelism` Flag

```
# Default: 10 concurrent operations
terraform apply

# Increase for faster applies (risk: API throttling)
terraform apply -parallelism=20

# Decrease for throttling-prone environments
terraform apply -parallelism=5
terraform apply -parallelism=2

# Serial execution (debugging only)
terraform apply -parallelism=1
```

🔵 When to Increase Parallelism

```
# Large applies with many independent resources
# Resources don't depend on each other → can be safely parallelized
# Your AWS account has high API rate limits (trusted advisor / business
support)

# Example: deploying 100 independent S3 buckets
# With -parallelism=10: 10 buckets at a time = 10 batches
# With -parallelism=25: 25 buckets at a time = 4 batches (faster)

terraform apply -parallelism=25
```

🔵 When to Decrease Parallelism

```
# 1. API throttling errors in plan/apply
# Error: RequestLimitExceeded / ThrottlingException
terraform apply -parallelism=3

# 2. IAM propagation issues
# Many IAM resources being created simultaneously can cause
# race conditions where policies aren't attached before roles are used
terraform apply -parallelism=5

# 3. Provider-specific limits
# Route53: very low rate limits → parallelism=2 recommended
```

```
# Lambda: concurrent creates can hit account limits

# 4. Debugging — easier to follow serial execution
terraform apply -parallelism=1
```

🔵 Parallelism vs Dependency Graph

```
Important: parallelism only affects INDEPENDENT resources
Dependent resources are ALWAYS sequential regardless of parallelism
setting

VPC → Subnet → EC2:
  Even with -parallelism=100
  VPC is created first, then subnet, then EC2
  Parallelism doesn't override dependencies

VPC → [Subnet A, Subnet B, Subnet C]:
  All three subnets are independent (same VPC dependency)
  parallelism=10: all three created simultaneously
  parallelism=1: created one at a time
```

🔵 Short Interview Answer

> "Terraform's `-parallelism` flag controls how many resource operations run concurrently — default is 10. Increase it when you have many independent resources and need faster applies, and your cloud account has sufficient API rate limits. Decrease it when you're hitting throttling errors — AWS IAM has lower rate limits (~20 calls/second) and Route53 is even lower (~5/second). Decreasing parallelism reduces concurrent API calls and prevents ThrottlingException errors. Key point: parallelism only affects independent resources — resources with dependencies are always executed sequentially regardless of the parallelism setting."

Topic 107: Provider Version Conflict Resolution

🔵 What It Is (Simple Terms)

Your root module requires AWS provider `~> 5.0` but a child module requires `~> 4.0`. Or two child modules require incompatible versions of the same provider. Terraform can't satisfy both constraints simultaneously.

🔵 How Conflicts Happen

```
# Root module: versions.tf
terraform {
  required_providers {
    aws = {
      source  = "hashicorp/aws"
      version = "~> 5.0"        # allows 5.x
    }
  }
}

# Child module A: modules/networking/versions.tf
terraform {
  required_providers {
    aws = {
      source  = "hashicorp/aws"
      version = ">= 4.0, < 5.0"   # allows 4.x only — CONFLICT with root
    }
  }
}

# terraform init error:
# Error: Inconsistent dependency lock file
# Provider "registry.terraform.io/hashicorp/aws" v5.1.0 does not match
# version constraint ">= 4.0, < 5.0" required by module.networking
```

🔵 Resolution Strategies

Strategy 1: Update the Child Module (Best)

```
# Update child module to accept newer versions
terraform {
  required_providers {
    aws = {
      source  = "hashicorp/aws"
      version = ">= 4.0, < 6.0"   # now accepts both 4.x and 5.x
    }
  }
}
# Modules should use permissive constraints (>= min, < next major)
```

Strategy 2: Constrain Root to Compatible Version

```
# If you can't update the child module, constrain root
terraform {
  required_providers {
    aws = {
      source  = "hashicorp/aws"
```

```
      version = ">= 4.0, < 5.0"   # downgrade root to match child
    }
  }
}
# Downside: you lose 5.x features
```

Strategy 3: Use a Newer Version of the Module

```
# Check if a newer version of the third-party module supports 5.x
module "networking" {
  source  = "terraform-aws-modules/vpc/aws"
  version = "5.0.0"   # newer module version supports AWS provider 5.x
}
```

Strategy 4: Fork and Fix the Module

```
# If the module is unmaintained, fork it and fix the constraint
module "networking" {
  source = "git::https://github.com/myorg/terraform-aws-vpc.git?ref=fix-
provider-v5"
}
```

🔵 Diagnosing Version Conflicts

```
# See all provider requirements across root + all modules
terraform providers

# Output:
# Providers required by configuration:
# .
# ├── provider[registry.terraform.io/hashicorp/aws] ~> 5.0
# └── module.networking
#     └── provider[registry.terraform.io/hashicorp/aws] >= 4.0, < 5.0

# The conflict is visible — root wants >= 5.0, module wants < 5.0
```

🔵 Terraform Core Version Conflicts

```
# Similar issue for Terraform Core version
# Child module: required_version = ">= 1.3.0, < 1.5.0"
# Your Terraform: 1.6.0
```

```
# Error: Module requires Terraform < 1.5.0 but you're running 1.6.0

# Resolution:
# 1. Update child module's required_version constraint
# 2. Use tfenv or terraform version manager to run older Terraform
# 3. Fork module and fix constraint
```

🔵 Short Interview Answer

> "Provider version conflicts happen when the root module and a child module have incompatible version constraints for the same provider. Terraform can't install a single provider version that satisfies both. Diagnose with `terraform providers` which shows all requirements across the tree. Resolution options in order of preference: update the child module to use a permissive constraint (`>= 4.0, < 6.0` accepts both 4.x and 5.x), use a newer version of the third-party module that already supports the newer provider, constrain the root module to match the child (downgrade), or fork the module and fix the constraint. Best practice for module authors: use permissive lower bounds (`>= X.0`) with an upper bound on major versions only — don't pin to exact versions in reusable modules."

📊 Category 11 Summary — Quick Reference Card

Topic	One-Line Summary	Interview Weight
88. Phantom Apply	Apply succeeds, resource doesn't exist — async API, provider bugs	⭐⭐⭐⭐
89. State Drift	Reality ≠ state — types of drift, refresh-only, detection tools	⭐⭐⭐⭐⭐
90. Concurrent Apply	State locking prevents corruption — DynamoDB, what happens without	⭐⭐⭐⭐
91. Cross-State Migration	`state mv` + config changes — zero destruction checklist	⭐⭐⭐⭐⭐
92. Stuck Lock Recovery	`force-unlock` — verify no apply running first	⭐⭐⭐⭐
93. Silent Drift	`ignore_changes`, `-refresh=false`, invisible resources	⭐⭐⭐⭐⭐
94. Perpetual Diff	JSON normalization, timestamp(), provider API format differences	⭐⭐⭐⭐⭐
95. Plan Passes Apply Fails	Time gap, IAM permissions, eventual consistency, quotas	⭐⭐⭐⭐⭐
96. Known After Apply	Cascades through count — use known values for count/for_each	⭐⭐⭐⭐

Topic	One-Line Summary	Interview Weight
97. Safe Resource Renaming	`moved` block — zero destroy rename	⭐⭐⭐⭐⭐
98. count to for_each	`state mv` per resource mapping old index → new key	⭐⭐⭐⭐⭐
99. Module Refactoring	`moved` blocks for root → child migration	⭐⭐⭐⭐
100. Module Breaking Changes	Pin exact versions, read changelog, `state mv` for structural changes	⭐⭐⭐⭐
101. Secret Commit Recovery	Rotate first, BFG purge second, pre-commit hooks to prevent	⭐⭐⭐⭐⭐
102. Sensitive Value Leaks	Mark all paths sensitive, avoid provisioner commands with secrets	⭐⭐⭐⭐
103. Retroactive Encryption	S3 cp object to itself with --sse flag — non-disruptive	⭐⭐⭐
104. Slow Plan Debugging	Large state, throttling, slow data sources — TF_LOG=DEBUG	⭐⭐⭐⭐⭐
105. Large State Management	Split by domain, 50-200 resources per state, migration path	⭐⭐⭐⭐⭐
106. Parallelism Tuning	Increase for speed, decrease for throttling — doesn't affect deps	⭐⭐⭐
107. Version Conflict Resolution	`terraform providers`, permissive module constraints	⭐⭐⭐⭐

🔑 The Golden Rules of Terraform Troubleshooting

```
Rule 1: ALWAYS back up state before any manual state operation
Rule 2: NEVER force-unlock without verifying no apply is running
Rule 3: ALWAYS run plan after any state manipulation
Rule 4: Rotate secrets BEFORE cleaning Git history
Rule 5: moved blocks > state mv > destroy/recreate (always prefer least
destructive)
Rule 6: Perpetual diff = provider normalizes differently than you write
Rule 7: Plan passes/apply fails = time gap, IAM, quotas, eventual
consistency
Rule 8: Silent drift = ignore_changes or -refresh=false hiding reality
Rule 9: Split state by team ownership + blast radius, not by convenience
Rule 10: Pin third-party module versions exactly in production
```

🎯 Category 11 — Top 5 Hardest Interview Questions

1. **"Walk me through migrating `count` resources to `for_each` without destroying anything"** — state mv mapping indexes to keys
2. **"Your plan shows no changes but someone tells you a security group rule is missing. How do you investigate?"** — silent drift, ignore_changes, refresh-only
3. **"terraform apply succeeded but the resource doesn't exist. What happened and what do you do?"** — phantom apply root causes and recovery
4. **"How do you move 20 resources from one state file to another without any downtime?"** — cross-state migration checklist
5. **"Your CI/CD plan takes 20 minutes. How do you debug and fix it?"** — slow plan root causes, TF_LOG=DEBUG, split state strategy

---

> **You've completed all 11 categories — 107 topics total.** Type any **topic number** to revisit, `quiz me` for interview practice, or `deeper` on any topic.